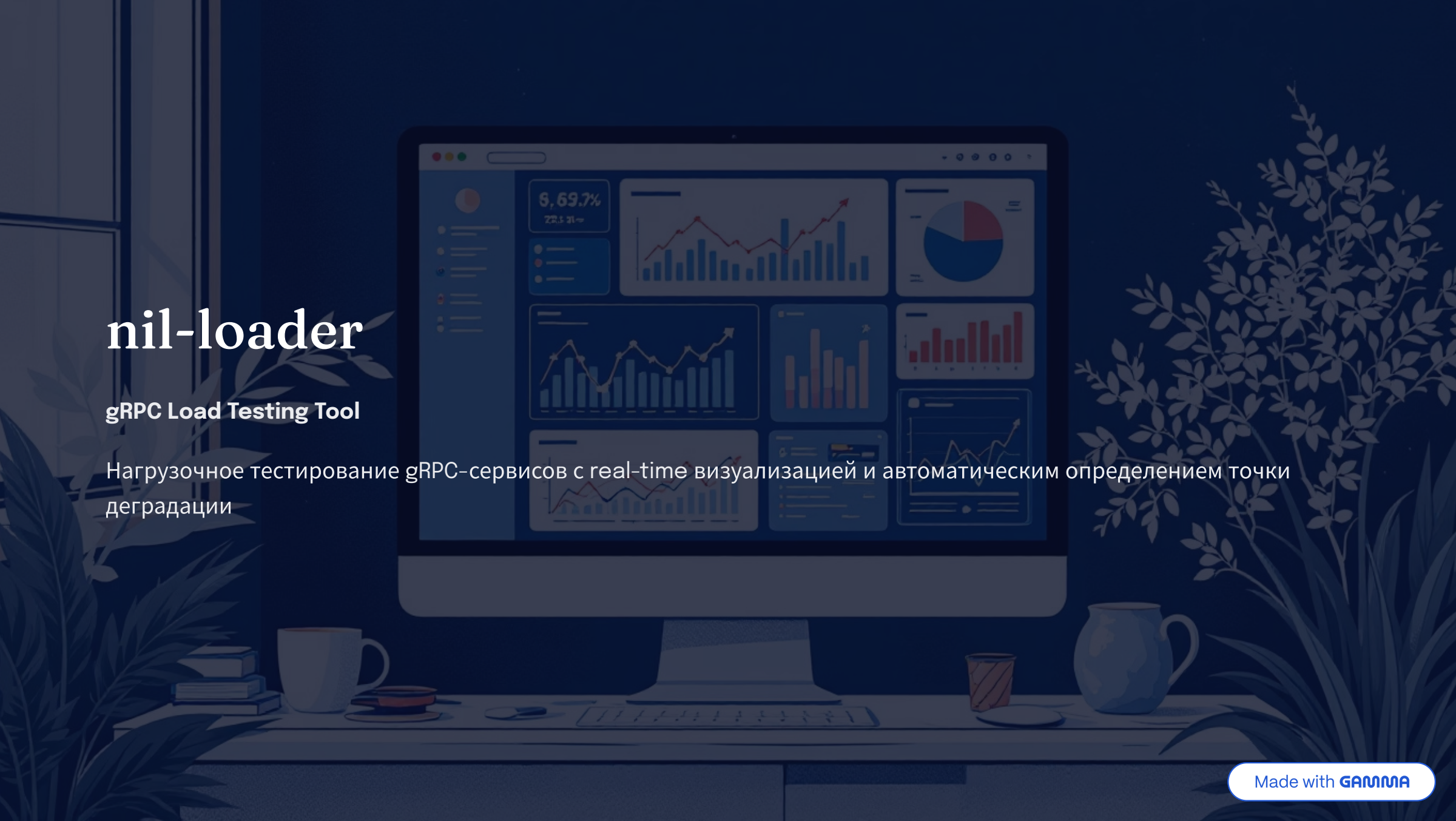
nil-loader

gRPC Load Testing Tool

Нагрузочное тестирование gRPC-сервисов с real-time визуализацией и автоматическим определением точки деградации



Made with GAMMA

Зачем нужно нагрузочное тестирование

Сервис работает стабильно на 10 пользователях, но что произойдёт при нагрузке в 10 000 пользователей? Нагрузочное тестирование отвечает на критически важные вопросы, которые определяют успешность вашего продукта в продакшне.

Максимальный RPS

Какой пиковой нагрузки выдерживает сервис без отказов?

Точка деградации

При какой нагрузке начинается нелинейный рост латентности?

Безопасный RPS

Какой уровень нагрузки гарантированно безопасен для продакшена?

Узкое горлышко

Где: CPU, память, сеть или база данных?

Существующие инструменты тестирования – такие как ghz, k6 и Locust – не всегда поддерживают gRPC «из коробки», требуют дополнительной кодогенерации или не предоставляют real-time визуализацию метрик.

Made with GAMMA

Базовые термины

Для понимания результатов нагрузочного тестирования важно знать ключевые метрики и термины.

Термин	Описание
RPS (Requests Per Second)	Количество запросов в секунду – основная метрика нагрузки
Latency	Время ответа на один запрос (в миллисекундах)
p50 / p95 / p99	Перцентили латентности. p99 = 99% запросов быстрее этого значения
Throughput	Реальная пропускная способность (фактический RPS)
Concurrency	Количество параллельных соединений / горутин
Error Rate	Процент неуспешных запросов
Точка деградации	RPS, при котором латентность или ошибки начинают расти нелинейно
HDR Histogram	Алгоритм для точного расчёта перцентилей без большого потребления памяти

Made with GAMMA

Динамическое управление нагрузкой

nil-loader предоставляет два режима управления нагрузкой для разных сценариев тестирования.

Constant RPS

Фиксированный уровень нагрузки на протяжении всего теста. Подходит для проверки стабильности при постоянной нагрузке.

- Задаётся фиксированный RPS
- Тест продолжается указанное время
- Worker Pool управляет concurrency

Ramping

Плавный рост нагрузки от Start RPS до End RPS за указанное время. Позволяет найти точку отказа сервиса.

- Линейный рост RPS в течение времени
- Видно момент, когда сервис перестаёт справляться
- Автоматическое определение точки деградации

Worker Pool с ограничением concurrency не «кладёт» машину-тестировщик, оставляя ресурсы для работы самого nil-loader и сбора метрик.

Made with GAMMA

Генератор данных

Тестирование с одинаковым payload приводит к кэшированию на стороне сервиса и даёт нереалистичные результаты. nil-loader решает эту проблему.

Шаблоны с Faker-токенами

Пользователь задаёт шаблон запроса с специальными токенами:

```
{
  "user_id": "{{faker.uuid}}",
  "name": "{{faker.name}}",
  "email": "{{faker.email}}",
  "phone": "{{faker.phone}}"
}
```

Каждый запрос получает уникальные данные. Доступные токены: `uuid`, `email`, `name`, `firstName`, `lastName`, `phone`, `username`, `sentence`, `timestamp`, `int`, `bool`, `safeAlphaNum`.

CSV-источник

Подстановка значений из CSV в запросы через `{{csv.column}}`.

Подходит для кастомных и реальных данных (например, `order_id`, `user_id`, способы оплаты) и точечной проработки нужных кейсов.

Made with GAMMA

Real-time дашборд

Четыре виджета обновляются каждую секунду через WebSocket, предоставляя непрерывное наблюдение за метриками.

Latency Chart

Три линии: p50, p95, p99. Видно «расслоение» при деградации – рост p99 быстрее p50 сигнализирует о проблемах.

Throughput

Target RPS vs Actual RPS. Расхождение = сервис «захлебнулся» и не справляется с нагрузкой.

Success Rate

Donut-график: OK vs ошибки. Плюс данные о деградации – момент, когда началась деградация.

Error Breakdown

Группировка по gRPC-кодам: Unavailable, DeadlineExceeded, Internal. Помогает диагностировать типы ошибок.

Error Log

Последние ошибки с кодом, сообщением и таймстемпом – быстрый доступ к деталям

Streaming Logs

Терминал (xterm.js) с логами движка в реальном времени – для отладки

Made with GAMMA

Автоматическое определение точки деградации

Nil-loader автоматически детектирует момент деградации сервиса, чтобы пользователь не делал это вручную.

Baseline p99

Первые 5 секунд – сбор нормальной латентности при низкой нагрузке

Проверка условий

Каждую секунду проверяется: p99 вырос > 2.5x от baseline? Или error rate > 15%?

Фиксация точки

Если условие выполнилось – фиксируем точку деградации и останавливаем тест

Degradation at

RPS, при котором началась деградация

Recommended

Безопасный RPS = 80% от точки деградации

Max stable

Максимальный RPS при стабильной работе

Made with GAMMA

Эффект от использования в цифрах

MVP версия nil-loader уже сегодня может дать ощутимый практический эффект для команд, которые тестируют gRPC-сервисы.

10

минут до первого отчёта

0

строк клиентского кода. Нет нагрузочных скриптов.

4

ключевые метрики в одном экране

1 = 3

1 прогон = 3 результата

2

инструмента в одном. Нагрузка и метрики

Made with GAMMA

Архитектура nil-loader

nil-loader построен как монолитное приложение с единой бинарной сборкой, где backend на Go включает в себя статические файлы UI.

Browser (UI)

HTTP/WS

nil-loader

Go backend с engine, telemetry и gRPC client

Target Service

gRPC сервис под тестированием

Engine

Rate limiter, worker pool с ограничением concurrency

Telemetry

HDR histogram, detection degradation

gRPC Client

Reflection, dynamic message

Все данные хранятся в памяти процесса. Через WebSocket реализована real-time передача метрик в UI для немедленного отображения на графиках.

Made with GAMMA

Dynamic gRPC без кодогенерации

Одна из ключевых фишек nil-loader – возможность тестировать gRPC-сервисы без предварительной генерации кода.

01

gRPC Reflection

Указываете адрес сервиса → автоматически получаете список сервисов и методов через gRPC Reflection Protocol

02

Загрузка .proto на лету

Вставляете proto-определение прямо в UI, без перекомпиляции nil-loader

03

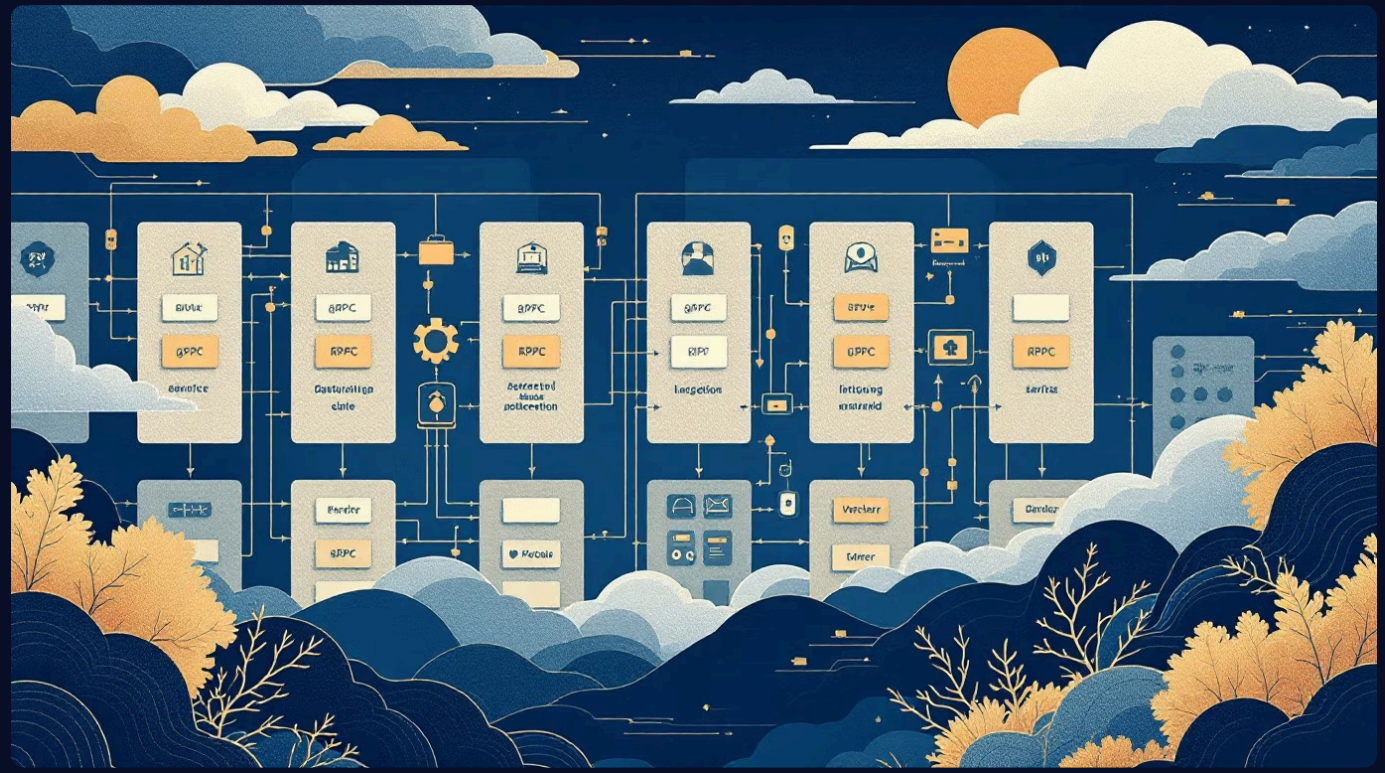
Dynamic Message

Формирование запросов без генерации кода с помощью jhump/protoreflect

> Пользователь просто вводит `target-service.ru:9999`, нажимает **Discover** и видит все доступные методы сервиса – никакой кодогенерации через protoc не требуется.

Made with GAMMA

Демо: тестируем Example Service



Тестовый gRPC-сервис

Встроен в проект для демонстрации функциональности. Метод ProcessUser принимает user_id, name, email, phone.

- До 800 RPS: ~5% ошибок, латентность 5–50ms
- Выше 800 RPS: ошибки растут экспоненциально, латентность квадратично
- Имитирует реальную деградацию сервиса

Made with GAMMA

Сценарий демо-теста

Discover via Reflection

Выбрать метод ProcessUser через gRPC Reflection

Вставить faker-шаблон

Генерация тестовых данных с помощью faker

Ramping 100 → 1500 RPS

Запуск теста с нарастанием нагрузки в течение 60 секунд

Наблюдать деградацию

Визуализация метрик в реальном времени на дашборде

Точка деградации ~800–900 RPS

Автоматическое определение критической нагрузки

Made with GAMMA

Телеметрия под капотом

Система сбора метрик обеспечивает точный мониторинг производительности в режиме реального времени

HDR Histogram

Точные перцентили p50, p95, p99 без хранения каждого значения. Диапазон 1μs – 30s с точностью 3 значащих цифры. Интервальная гистограмма сбрасывается каждую секунду.

Error Classifier

Автоматическая группировка по gRPC status codes: Unavailable, DeadlineExceeded, Internal, Unknown. Кумулятивные и интервальные счётчики ошибок.

Snapshot (1 сек)

Структура: timestamp, p50, p95, p99, targetRPS, actualRPS, success, errors, errorBreakdown. Посекундная детализация метрик.

Made with GAMMA

Стек технологий

Выбранные компоненты обеспечивают высокую производительность и простоту интеграции

Компонент	Технология	Обоснование
Backend	Go 1.25	Высокая производительность, простая конкуррентность через goroutines, статическая сборка
gRPC Client	jhump/protoreflect	Dynamic Client без кодогенерации, работает с reflection и .proto файлами
Protobuf	gRPC / Protobuf	Официальный стандарт RPC, совместимость с экосистемой
Metrics	HdrHistogram	Точные перцентили без хранения всех значений, подходит для high-load
WebSocket	gorilla/websocket	Надёжная реализация, удобная интеграция с HTTP-сервером
Frontend Charts	Chart.js 4	Интерактивные графики для дашборда MVP
Terminal	xterm.js	Стандарт для web-терминалов, дружелюбен к потоковым логам
Сборка	go:embed	Один бинарник без внешних зависимостей

Минимальные зависимости, максимальное использование стандартной библиотеки Go

Made with GAMMA

Структура проекта

```
nil-loader/
├── cmd/
│   ├── server/main.go        — UI + API сервер
│   └── testservice/main.go   — тестовый gRPC-сервис
├── internal/
│   ├── model/types.go        — доменные модели
│   ├── engine/engine.go      — движок нагрузки
│   ├── grpcclient/           — reflection, proto, dynamic caller
│   ├── payload/generator.go  — faker + CSV + шаблоны
│   ├── telemetry/
│   │   ├── collector.go      — HDR Histogram + метрики
│   │   └── degradation.go    — детектор деградации
│   └── api/
│       ├── handler.go        — REST API
│       └── ws.go             — WebSocket
├── proto/example/            — proto + сгенерированный код
└── web/index.html            — весь UI (embedded)
```

Проект оформлен как стандартный Go-проект с логичной структурой, включает README, лицензию MIT

Made with GAMMA

Функциональность MVP

1

Запуск нагрузочного теста

Тестирование одного gRPC-метода (unary) с режимами Constant и Ramping

2

Обнаружение сервисов

Автоматическое обнаружение через gRPC Reflection и загрузка .proto файлов

3

Генерация данных

Faker-шаблоны и CSV-источник для создания тестовых данных

4

Real-time дашборд

Визуализация p50/p95/p99, Target vs Actual RPS, Success Rate, ошибки по gRPC-кодам

5

Автодетектор деградации

Расчёт точки отказа и рекомендуемого RPS

6

Экспорт отчёта

JSON-отчёт по результатам теста для анализа

Made with GAMMA

Ограничения и интеграции

Текущие ограничения

- Один gRPC-метод за раз
- Только unary-методы
- Данные в памяти
- Нет истории тестов
- Один инстанс

CI/CD интеграция (to be)

nil-loader можно запускать в пайплайне (GitLab CI / GitHub Actions) как отдельный шаг для автоматической проверки SLA.

- Поднять тестовый сервис
- Запустить nil-loader
- Выполнить Ramping-тест
- Проверить метрики

Будущее развитие

QA Profile

QA могут настраивать платформу под себя. Свои сервисы, свои шаблоны.

История тестов

Сохранение и сравнение результатов

HTTP/REST

Поддержка помимо gRPC

Streaming-методы

Client/server/bidi streaming

CI/CD интеграция

CI/CD интеграция с автоматической проверкой SLA

Made with GAMMA

Итоги

Real-time UI Графики, логи, ошибки через WebSocket	**Умная нагрузка** Constant / ramping + worker pool
Уникальные данные Faker + CSV шаблоны	**Автодетекция деградации** Точка отказа, рекомендуемый RPS
Точная телеметрия HDR Histogram, посекундные snapshots	**Без кодогенерации** Reflection + dynamic messages

Один бинарник. Ноль конфигурации. Открыл браузер — тестируешь.

Made with GAMMA

О проекте

Проект разработан командой nil-team

Участники:

- Владимир Гальцев
- Никита Шморин
- Александр Семёнов
- Александр Вяткин

Made with GAMMA